Amrita Vishwa Vidyapeetham, Coimbatore
Department of Computer Science and Engineering

Software Requirements Specification

for

CITYZEN: AI-Powered Civic Intelligence

Course: 23CSE498 – Project Phase 2
Team Number: 60

S.No	Register Number	Student Name
1	CB.SC.U4CSE23222	Keshav S
2	CB.SC.U4CSE23254	Vimal HariHar SK
3	CB.SC.U4CSE23406	Arun N
4	CB.SC.U4CSE23462	KOKUL M
5	CB.SC.U4CSE23648	Ravula Akshith

Guide: **Dr. Prathilothamai M**, Department of CSE

Document Version: 2.0
October 22, 2026

Prepared in accordance with IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

Contents

1 Introduction **2**
1.1 Purpose . . . 2
1.2 Scope . . . 2
1.3 Document Conventions . . . 2
1.4 Intended Audience and Reading Suggestions . . . 3
1.5 Definitions and References . . . 3

2 Overall Description **3**
2.1 Product Perspective . . . 3
2.2 Product Functions . . . 3
2.3 User Classes and Characteristics . . . 5
2.4 Operating Environment . . . 5
2.5 Design and Implementation Constraints . . . 5
2.6 Assumptions and Dependencies . . . 6

3 External Interface Requirements **6**
3.1 User Interfaces . . . 6
3.2 Hardware Interfaces . . . 6
3.3 Software Interfaces . . . 7
3.4 Communications Interfaces . . . 7

4 System Features and Functional Requirements **7**
4.1 User Registration and Authentication . . . 7
4.2 Complaint Submission and Retrieval . . . 8
4.3 Image Validation and Explainable Segmentation . . . 9
4.4 Text Authenticity and Emotion Analysis . . . 9
4.5 Multilingual Speech-to-Text . . . 10
4.6 Location Intelligence and Location Privacy . . . 10
4.7 Priority Scoring Engine (AHP) . . . 11
4.8 Duplicate Detection and Voting . . . 11
4.9 Pothole Geometry Estimation . . . 12
4.10 Administrative Workflows . . . 12
4.11 Transparency, Maps and Statistics . . . 13
4.12 Civic Chatbot . . . 13
4.13 Feedback, Localisation and Auxiliary Services . . . 13

5 Non-Functional Requirements **14**
5.1 Performance . . . 14
5.2 Security and Privacy . . . 14
5.3 Reliability and Availability . . . 15
5.4 Usability . . . 15
5.5 Maintainability and Testability . . . 15
5.6 Portability . . . 15

6 Other Requirements **16**
6.1 Glossary . . . 16
6.2 Data Dictionary . . . 16
6.3 Open Issues and Future Enhancements . . . 17
6.4 References . . . 17

List of Figures

2.1 CITYZEN system architecture 4

List of Tables

2.1 User Classes and Characteristics 5
2.2 Operating Environment 5
3.1 External Software Interfaces 7
4.1 AHP-Derived Weights Used by the Priority Engine 11
6.1 Glossary of Terms 16
6.2 CITYZEN Data Dictionary 16
6.3 Implementation Status of Incomplete or Unverified Items 17

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of **CITYZEN**, an AI-assisted civic complaint management platform, as it is *actually implemented*. Every requirement was derived from the implemented source code, database migrations, configuration files and tests. Where the intended behaviour and the implemented behaviour differ, the requirement carries an explicit status tag (Section 1.3).

1.2 Scope

CITYZEN lets citizens report civic infrastructure problems from a mobile or web client using a photo, a typed or spoken description, and their location. The backend validates the photo with a segmentation model, checks the description for authenticity and emotional urgency, scores the complaint's priority with an Analytic Hierarchy Process (AHP) model that also considers nearby critical facilities, and merges nearby duplicates into votes on an existing complaint. Administrators work from a priority queue, update complaint status or a three-stage workflow, and inspect AI explanations. Citizens can vote on complaints, follow their own reports, use a multilingual chatbot and view a public transparency dashboard.

The product consists of a Node.js/Express REST backend, a React web application, an Expo/React Native mobile application and an optional Python helper service, all developed together. The system has also been referred to as *UrbanPulse*, *CivicStack* and *Civic Rezo*. **Not in scope** (no implementation exists): SMS, e-mail or push notifications; social-media cross-validation of complaints; integration with municipal ERP or ticketing systems; and real officer or contractor management (Section 6.3).

1.3 Document Conventions

- **Shall** denotes a mandatory requirement. Functional requirements are numbered **FR-4.*x.y*** by feature (Chapter 4); non-functional requirements are numbered **NFR-5.*x.y*** (Chapter 5).
- A requirement without a tag is implemented. Exceptions are tagged *[Partial]* (present with a documented gap), *[Not implemented]* (required or implied but absent) or *[Unverified]* (depends on artefacts that are not available for review).
- API endpoints are written relative to the backend base URL, e.g. `POST /api/complaints/submit`; identifiers, columns and environment variables are set in `monospace`.

- Scores are in the range $[0, 1]$ unless a 0–100 scale is stated.

1.4 Intended Audience and Reading Suggestions

The document is written for the project team and future maintainers, the faculty guide and review panel, and testers. Reviewers should read Chapters 2 and 4; developers should also read Chapter 3 and the data dictionary (Section 6.2); testers can use the requirement IDs in Chapters 4 and 5 as test-case references. Section 6.3 lists every known gap in one place.

1.5 Definitions and References

Terms and acronyms are defined in the glossary (Table 6.1); cited standards and literature are listed in Section 6.4.

2. Overall Description

2.1 Product Perspective

CITYZEN is a new, self-contained system with three deployable units and one optional helper:

- **Mobile app** (`frontend/app`): Expo / React Native app with citizen and administrator screens, camera capture, device-tilt sampling, GPS and voice input.
- **Web app** (`frontend/urbanpulse-web`): React 18 + Vite single-page application with a citizen portal and an administrator console.
- **Backend** (`backend`): Node.js/Express server exposing 18 mounted REST routers plus a health check. It is the only component that writes to the database and the only caller of the AI services, except for the client-side image upload to Cloudinary.
- **Python helper** (`backend/python_services`): an optional Flask service on port 5001 that forwards sentiment requests to the hosted DistilBERT SST-2 model on Hugging Face.

Data is stored in a Supabase-hosted PostgreSQL database accessed through `supabase-js`. Figure 2.1 shows the layered architecture.

Where the diagram and the source code differ, this SRS follows the code. Speech-to-text supports six languages, not ten (FR-4.5.2). The officer and contractor stores are static lists rather than database tables (FR-4.10.6). The vote input to the AHP blend is always zero in stored scores (FR-4.7.5). The mobile app declares Expo SDK 57 in `frontend/app/package.json`.

2.2 Product Functions

- Account registration and JWT-based login for citizens and administrators (§4.1).
- Multi-step complaint submission with category, photo, description and location (§4.2).
- Photo validation and explainable segmentation using Roboflow SAM3 workflows (§4.3).
- Description authenticity and emotion analysis (§4.4); six-language speech-to-text (§4.5).
- Facility-proximity scoring with privacy levels (§4.6); AHP priority scoring (§4.7).
- Duplicate merging and voting (§4.8); pothole size and depth estimation (§4.9).
- Administrator dashboards, priority queue, status and workflow management (§4.10).
- Public transparency statistics and heat maps (§4.11); civic chatbot (§4.12); feedback, localisation and weather (§4.13).

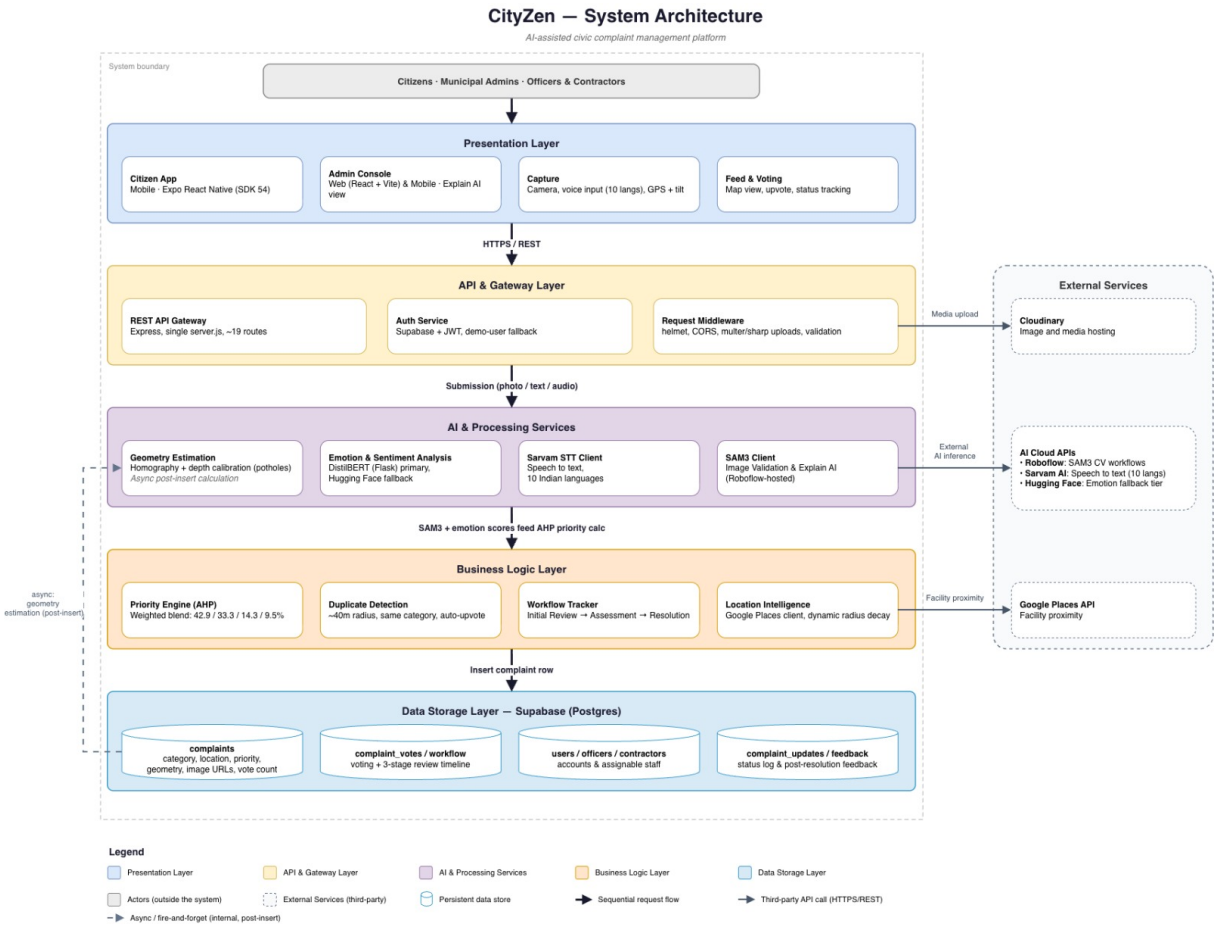


Figure 2.1: CITYZEN system architecture

2.3 User Classes and Characteristics

Table 2.1: User Classes and Characteristics

User class	Characteristics and permitted actions	Evidence
Citizen	Registers with `userType = citizen`. Submits complaints, votes, views feed, map, personal reports, transparency data, chatbot and feedback. Uses English or one of five Indian languages.	`routes/auth.js`, web `App.jsx` route guards
Administrator	Registers with `userType = admin` through the self-service admin sign-up page. Uses dashboards, priority queue, complaint detail with Explain AI and geometry, status/workflow updates, citizen management.	`routes/adminMinimal.js`, `routes/adminComplaints.js`
Anonymous / guest	Has no token. The backend accepts complaint submission (stored under a shared demo account), guest votes (per device) and feedback. Web citizen pages require login.	`middleware/auth.js`, `routes/guest-votes.js`
Officer / contractor	Not system users: no accounts or login. They appear only as two static entries each returned by the admin API.	`adminMinimal.js` `/officers`, `/contractors`

2.4 Operating Environment

Table 2.2: Operating Environment

Component	Environment (as configured)
Backend	Node.js 18 or later (CI uses Node 20); Express 4; deployed as a Render web service (`backend/render.yaml`: region Singapore, free plan, port 3001).
Web app	Modern desktop or mobile browser with Geolocation and MediaRecorder APIs; static build by Vite, served with an SPA rewrite (`vercel.json`); default API base URL from `VITE_API_BASE_URL`, else the Render deployment.
Mobile app	Android or iOS through Expo (SDK 57, React Native 0.86); requires camera, location, microphone and motion-sensor permissions.
Database	Supabase PostgreSQL; schema extended by the migrations in `backend/database`.
Python helper	Python 3.10 or later, Flask 3.1, `huggingface_hub`; optional.

2.5 Design and Implementation Constraints

- The backend connects with the Supabase *anon* key (`SUPABASE_ANON_KEY`); data protection therefore depends on database policies and on the backend's own checks.
- Data definitions for the base tables (`users`, `complaints`, `complaint_votes`, `complaint_updates`, `complaint_workflow`) are not versioned; only incremental migrations are. Section 6.2 reconstructs them from code.
- Photo validation accepts only an already-hosted HTTPS image URL; base64 input is rejected.
- The image taxonomy is fixed by the Roboflow workflow to seven classes; speech-to-text and UI localisation are fixed to six languages.
- Camera intrinsics are unavailable from `expo-image-picker`, so pothole geometry uses documented assumed camera constants (FR-4.9.3).
- Score columns are stored with two decimal places; the backend clamps them to $[0, 9.99]$.

2.6 Assumptions and Dependencies

- Roboflow, Hugging Face, Sarvam AI, Groq, Google Places, Cloudinary, OpenStreetMap Nominatim and Open-Meteo are reachable, and the corresponding keys are set in `backend/.env`. Missing keys degrade features as described in NFR-5.3.1.
- The optional migrations have been applied; until they are, the backend omits language, geometry and per-device vote data (NFR-5.3.2).
- Device GPS accuracy is adequate for the 40 m duplicate radius; the backend has no independent location check.
- Pothole photos are taken with a handheld phone at roughly 1.4 m height, as assumed by the geometry model.
- Hosting on free-tier services is assumed acceptable; no availability target is defined.

3. External Interface Requirements

3.1 User Interfaces

- **Web citizen portal** (`/citizen/*`): feed with voting, dashboard, five-step complaint form (classification, evidence, narrative, geodata, audit and submit), Leaflet map with heat layer, personal reports with stage timeline, complaint detail, chatbot, feedback, transparency dashboard; a language switcher on every page.
- **Web admin console** (`/admin/*`): dashboard, complaint list, complaint detail with status control, Explain AI modal and geometry card, priority queue, complaint map, citizen management and citizen details.
- **Mobile app**: welcome and separate citizen/admin authentication screens; citizen dashboard, multi-step submission, feed, map, complaint detail, transparency, personal reports, chatbot and feedback; admin dashboard, priority queue, complaint details with three-stage workflow, complaint map and citizen management.

3.2 Hardware Interfaces

There are no special server hardware requirements. On clients the system uses the rear camera (photo evidence), GPS or location services (geotagging), the microphone (voice dictation) and, on mobile only, the motion sensor through `expo-sensors` to sample camera tilt during capture.

3.3 Software Interfaces

Table 3.1: External Software Interfaces

Interface	Purpose in CITYZEN	Configuration
Supabase PostgreSQL	Persistent store for users, complaints, votes, updates, workflow, feedback	`SUPABASE_URL`, `SUPABASE_ANON_KEY`
Roboflow Serverless Workflows	SAM3 validation and annotated segmentation; separate SAM3 + depth workflow for pothole geometry	`ROBOFLOW_API_KEY`, `ROBOFLOW_WORKSPACE`, `ROBOFLOW_WORKFLOW`, `ROBOFLOW_GEOMETRY_WORKFLOW`
Hugging Face Inference API	Multilingual zero-shot authenticity (mDeBERTa-v3 XNLI); sentiment fallback models	`HUGGING_FACE_API_KEY`, `HUGGINGFACE_API_TOKEN`, `HF_AUTHENTICITY_MODEL`
DistilBERT helper (Flask)	Sentiment via hosted DistilBERT SST-2	`DISTILBERT_SERVICE_URL` (default port 5001)
Sarvam AI	Speech-to-text, model `saarika:v2.5`	`SARVAM_API_KEY`
Groq (OpenAI-compatible)	Grounded chatbot replies	`GROQ_API_KEY`, `GROQ_MODEL`
Google Places API	Nearby critical-facility search (New API, legacy fallback)	`GOOGLE_PLACES_API_KEY`
Cloudinary	Image hosting (client-side unsigned upload); image deletion endpoint	`CLOUDINARY_*`
OpenStreetMap	Reverse geocoding (Nominatim, web) and map tiles (Leaflet)	public HTTPS
Open-Meteo	Current weather and daily forecast	public HTTPS

3.4 Communications Interfaces

- All client–backend and backend–service traffic is HTTP(S) with JSON bodies, except audio upload (`multipart/form-data`, field `audio`).
- Authenticated requests carry `Authorization: Bearer <JWT>`.
- In production, CORS admits only the configured origins (local development ports, the Vercel deployment and its preview URLs, and `FRONTEND_URL`); requests without an `Origin` header (mobile clients) are accepted.
- `GET /health` returns `status: OK` with a timestamp for liveness checks.

4. System Features and Functional Requirements

4.1 User Registration and Authentication

Account creation and login against the application's own `users` table (Supabase Auth is not used). **Priority:** High. **Status:** Implemented, except server-side role enforcement.

FR-4.1.1: `POST /api/auth/signup` shall require a valid e-mail, a password of at least 6 characters, a trimmed full name of at least 2 characters, a phone number of at least 10 characters and `userType` $\in$ {citizen, admin}, and shall return HTTP 400 with the validation errors otherwise.

FR-4.1.2: The system shall reject sign-up for an existing e-mail (HTTP 400) and otherwise store the user with a bcrypt password hash (cost 10) and `is_active = true`.

FR-4.1.3: On successful sign-up or login the system shall return a JWT signed with `JWT_SECRET` that contains `userId`, `email` and `userType` and expires after 7 days, together with the profile without the password.

FR-4.1.4: Login shall succeed only for active users with a matching password; any other case shall return HTTP 401 "Invalid email or password". Successful login shall update `last_login`.

FR-4.1.5: `GET /api/auth/profile` shall return the profile of the bearer-token owner, HTTP 401 for a missing or invalid token and HTTP 404 if the user no longer exists.

FR-4.1.6: For every request the backend shall attach the identity in a valid bearer token; requests with a missing or invalid token shall continue as anonymous rather than be rejected.

FR-4.1.7: The web and mobile clients shall show citizen and admin screens only to users of the matching type; the web client shall keep the token and profile in `localStorage`.

FR-4.1.8: The backend shall restrict administrator endpoints to admin users. The `authorizeUserType` middleware exists but is not applied to any route. *[Not implemented]*

4.2 Complaint Submission and Retrieval

A citizen files a complaint through a guided form; the backend validates, scores and stores it. **Priority:** High. **Status:** Implemented.

FR-4.2.1: The clients shall require exactly one of seven categories: `pothole`, `fallen_tree`, `garbage_dumping`, `stray_cattle`, `fallen_electric_pole`, `concrete_structure_damage`, `road_waterlogging`.

FR-4.2.2: The web form shall advance past the evidence step only after a validated photo or an explicit skip, and past the narrative step only with a title of at least 3 and a description of at least 10 characters and no hard-blocked authenticity result (FR-4.4.5).

FR-4.2.3: `POST /api/complaints/submit` shall return HTTP 400 `MISSING_REQUIRED_FIELDS` when title, description, category or location data is missing.

FR-4.2.4: The complaint owner shall be the authenticated user; otherwise a UUID supplied in the request; otherwise a shared demo account (`demo@civicrezo.org`), created if absent.

FR-4.2.5: If an image URL is supplied without `imageValidation.allowUpload = true`, the system shall return HTTP 400 `IMAGE_CATEGORY_MISMATCH` with the detected class.

FR-4.2.6: An accepted complaint shall be stored with status `pending`, coordinates, address (or "lat, lng" to 4 decimals), priority, location, emotion, AI-confidence (client-supplied `modelConfidence`, else 0.5) and text-authenticity scores, flag and reason, language (default `en`) and image URL list.

FR-4.2.7: `verification_status` shall be `flagged_for_review` when the text is flagged, else `verified` when the photo was validated, else `unverified`.

FR-4.2.8: After insertion the system shall add a `complaint_updates` entry "Complaint submitted", register the owner's upvote, and start pothole geometry estimation without awaiting it.

FR-4.2.9: The response shall include the complaint ID, priority score (0–100) and level, score breakdown and reasoning, content-analysis results and level-specific next-step messages.

FR-4.2.10: The insert shall contain only columns present in the live table, and score fields

shall be clamped to $[0, 9.99]$ with two decimals.

FR-4.2.11: The system shall provide complaint retrieval: paginated `GET /api/complaints` (status, category, user, admin, sort and radius filters), `GET /api/complaints/all`, `GET /api/complaints/:id` with the caller's vote state, and `GET /api/complaint-details/nearby`.

FR-4.2.12: `GET /api/complaints/personal-reports` shall require authentication (HTTP 401 otherwise) and return the caller's complaints with a five-stage tracking timeline and status counts.

4.3 Image Validation and Explainable Segmentation

Photos are checked against the selected category by the Roboflow "CityZen SAM3" workflow. **Priority:** High. **Status:** Implemented; server trusts the client's validation flag.

FR-4.3.1: Clients shall require a category before a photo, upload the photo to Cloudinary and send the returned HTTPS URL for validation.

FR-4.3.2: `POST /api/image-analysis/validate-image` shall take `imageUrl` and an optional `category`; a missing URL, or base64 data alone, shall return HTTP 400.

FR-4.3.3: The workflow call shall reject non-HTTPS URLs, time out after 20 s and be retried at most twice with exponential back-off (0.5 s, 1 s), only on network errors or HTTP 5xx responses.

FR-4.3.4: When the workflow returns no detections, the result shall be `allowUpload = false` with "No valid civic issue detected in image."

FR-4.3.5: When a known category is given, the photo shall be accepted only if a detection of that class exists; its confidence becomes the image confidence. Otherwise the result shall name the detected and the selected issue. Without a category, the highest-confidence detection is used.

FR-4.3.6: Workflow failures shall produce `success = false`, `allowUpload = false` and the error message instead of an exception.

FR-4.3.7: `POST /api/image-analysis/explain` shall return a base64 annotated image, the detections and a plain-language explanation. For potholes it shall overlay only pothole masks from the geometry workflow, falling back to the standard annotated image.

FR-4.3.8: `GET /api/image-analysis/categories` shall list the seven detectable classes.

FR-4.3.9: The submission endpoint shall verify the photo itself rather than trust the client-supplied `imageValidation` object. *[Not implemented]*

4.4 Text Authenticity and Emotion Analysis

Descriptions are scored for topical match and genuineness, and for emotional urgency. **Priority:** High. **Status:** Implemented.

FR-4.4.1: `POST /api/complaints/analyze-text` shall return emotion and authenticity results (HTTP 400 `MISSING_DESCRIPTION` if empty). The web form shall call it 800 ms after typing stops once the description has at least 15 characters.

FR-4.4.2: The rule-based score shall blend category-keyword match and text quality with AHP weights 0.636 / 0.364.

FR-4.4.3: The model score shall come from two zero-shot calls to `MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7` (category labels plus "none of these"; genuine vs. spam vs. gibberish), combined 0.636 / 0.364. The final score shall be $0.70 \times$ model $+ 0.30 \times$ rule-based.

FR-4.4.4: If the model is unavailable (no key, error, 7 s per call or 8 s overall timeout), the rule-based score shall be used alone.

FR-4.4.5: A score below 0.25 shall reject submission with HTTP 400 `DESCRIPTION_CATEGORY_MISMATCH`, reasons and a suggested category; a score from 0.25 to below 0.45 shall accept the complaint with `content_flagged = true`.

FR-4.4.6: Emotion analysis shall detect Hindi, Tamil, Telugu, Kannada or Malayalam by Unicode script (else English). Hindi and Tamil use multilingual keyword analysis; other text tries the DistilBERT helper, then four Hugging Face sentiment models, then keyword analysis.

FR-4.4.7: The emotion score shall combine urgency, concern, anger and frustration with AHP weights 0.474 / 0.263 / 0.158 / 0.105, be raised 20% when concern exceeds 0.3, scaled by the category-tier multiplier (Table 4.1), boosted for safety keywords and capped at 1.0. On failure it shall be 0.5.

4.5 Multilingual Speech-to-Text

Citizens may dictate descriptions and chatbot messages. **Priority:** Medium. **Status:** Implemented (Sarvam provider only).

FR-4.5.1: `POST /api/transcribe/audio` shall accept one file in field `audio` with an `audio/*` MIME type up to 50 MB; a missing file shall return HTTP 400.

FR-4.5.2: The language shall be one of `en`, `hi`, `ta`, `te`, `ml`, `kn` (sent to Sarvam as *xx*-IN); any other value shall fall back to English.

FR-4.5.3: Non-WAV audio shall be converted with ffmpeg to 16 kHz mono 16-bit PCM WAV and sent to Sarvam `saarika:v2.5` with a 45 s timeout.

FR-4.5.4: The transcript shall be returned in the spoken language without translation; on failure the system shall return HTTP 502 advising the user to type instead.

FR-4.5.5: Uploaded and converted audio files shall be deleted after every request.

FR-4.5.6: Voice dictation shall be offered in the web complaint form and chatbot (via MediaRecorder) and in the mobile complaint form.

FR-4.5.7: A second STT provider (Qwen) shall be selectable; the provider is a stub that throws. *[Not implemented]*

4.6 Location Intelligence and Location Privacy

Location sets the infrastructure component of priority and is subject to citizen-chosen privacy. **Priority:** High. **Status:** Implemented; web privacy rounding missing.

FR-4.6.1: Clients shall capture GPS coordinates (mobile: `expo-location`; web: Geolocation API, high accuracy, 10 s timeout); the web client shall reverse-geocode with Nominatim.

FR-4.6.2: The citizen shall choose privacy level exact, street or area; the mobile app shall round coordinates to 4 decimals (accuracy 25 m) for street and 3 decimals (150 m) for area.

FR-4.6.3: The web client shall apply the same rounding; it currently sends unrounded coordinates with only the privacy label. *[Partial]*

FR-4.6.4: The backend shall classify the area (dense urban, urban, suburban, rural, unknown) with a 1 km facility probe and set a base search radius of 0.8–3.5 km, widened (up to 1.5×, capped) for fewer than 10 facilities, narrowed (0.7×, minimum 500 m) above 50, plus up to 500 m for poor GPS accuracy.

FR-4.6.5: The backend shall query Google Places (New) Nearby Search for eight facility types weighted per Table 4.1, falling back to the legacy Nearby Search on non-fatal errors.

FR-4.6.6: Each facility shall score $e^{-\ln 2 \cdot d/d_{1/2}}$ with $d_{1/2}$ half the type's configured radius; scores combine with density and diversity bonuses, a complaint-type multiplier and a privacy factor (exact 1.0, street 0.95, area 0.90), capped at 1.

FR-4.6.7: If location scoring fails, the comprehensive priority shall be 0.1 with the error as reasoning; without coordinates a category-tier fallback $0.2 + 0.7 \cdot t/9$ shall be used.

4.7 Priority Scoring Engine (AHP)

All weights are defined once in `services/priorityConfig.js` and derived by `services/ahp.js`. **Priority:** High. **Status:** Implemented; vote signal inactive.

Table 4.1: AHP-Derived Weights Used by the Priority Engine

Group (1–9 judgments)	**Derived weight or multiplier**
Top-level signals	Image evidence 9 → 0.429; infrastructure proximity 7 → 0.333; text emotion 3 → 0.143; community votes 2 → 0.095
Facility types	Hospital 9, fire station 9 → 0.191 each; police 7 → 0.149; school 6 → 0.128; transit station 5, pharmacy 5 → 0.106 each; government 4 → 0.085; bank 2 → 0.043
Category tiers (multiplier $1 + 0.9\,t/9$)	Life-threatening 9 → 1.90 (`fallen_electric_pole`); infrastructure integrity 6 → 1.60 (`pothole`, `fallen_tree`, `concrete_structure_damage`, `road_waterlogging`); basic services 4 → 1.40 (`garbage_dumping`, `stray_cattle`); unmapped categories 1 → 1.10
Authenticity	Category match 7 → 0.636; text quality 4 → 0.364; model 7 → 0.70 vs. rule-based 3 → 0.30

FR-4.7.1: Weights shall be derived from 1–9 judgments by the normalised-column-average AHP method, with the consistency ratio computed and judged acceptable at ≤ 0.10.

FR-4.7.2: The score shall be $0.429\,I + 0.333\,F + 0.143\,E + 0.095\,V$ on a 0–100 scale, where I is validated image confidence $\times 100$ (0 without a photo), F the facility score, E the emotion score and $V = 100(1 - e^{-v/5})$ for v votes. It shall be multiplied by 1.2 for `in_progress` or 0.5 for `completed`, capped at 100 and stored on a 0–1 scale.

FR-4.7.3: Priority levels shall be CRITICAL (≥ 0.8), HIGH (≥ 0.6), MEDIUM (≥ 0.4), LOW (≥ 0.2) and MINIMAL.

FR-4.7.4: `POST /api/complaints/calculate-priority` shall preview the score (0–100), level, reasoning, breakdown and processing time; it needs category and coordinates (else HTTP 400).

FR-4.7.5: Priority shall be recomputed when votes or status change. It is computed only at submission with $v = 0$ and status `pending`, so the vote and status terms never affect stored scores. *[Not implemented]*

4.8 Duplicate Detection and Voting

Nearby reports of the same issue become votes; citizens can vote on any complaint. **Priority:** High. **Status:** Implemented; guest de-duplication depends on a migration.

FR-4.8.1: Before text analysis and scoring, the system shall look for an open complaint (not `resolved` or `rejected`) of the same category within 40 m, using a bounding-box query and Haversine distance, and select the nearest.

FR-4.8.2: If one is found, no complaint shall be created; the submitter's upvote shall be added to it (no-op if already voted) and the response shall have `duplicate = true`, the existing complaint, `voteCount`, `alreadyVoted` and an explanation.

FR-4.8.3: A failed duplicate lookup shall be treated as "no duplicate".

FR-4.8.4: `POST /api/complaints/vote` shall require authentication (HTTP 401), a `complaintId` (HTTP 400) and an existing complaint (HTTP 404), and shall toggle the caller's upvote.

FR-4.8.5: After every vote change the system shall recount the upvote rows and persist the result in `vote_count`.

FR-4.8.6: `POST /api/guest-votes` shall toggle a vote per `deviceId`; this works only once `complaint_votes.device_id` exists, otherwise every request adds a vote. *[Partial]*

FR-4.8.7: `GET /api/complaints/vote/status` shall return, for authenticated users, which of the listed complaints they have voted for.

4.9 Pothole Geometry Estimation

Single-photo estimation of pothole footprint and depth, shown to administrators. **Priority:** Medium. **Status:** Implemented.

FR-4.9.1: Estimation shall run asynchronously after insertion and only when the category or detected class is `pothole`; other complaints shall be marked `not_applicable`.

FR-4.9.2: A missing image or an empty geometry-workflow result shall set status `failed` with the reason in `geometry_error`.

FR-4.9.3: Rim points of the highest-confidence instance shall be projected onto the ground plane with a pinhole model (camera height 1.4 m, field of view 70° × 55°, tilt = device tilt if within −10° to 100°, else 35°).

FR-4.9.4: Depth shall be derived by regressing depth-model output against slant distance over the rim points. A small ring sample (<50 px) or sparse rim (<10 points) shall halve the confidence, a poor fit ($R^2 < 0.3$) scale it by 0.6 and an assumed tilt by 0.7; depths above 60 cm shall be flagged.

FR-4.9.5: Width, length, area, depth, confidence, method version, assumptions, status and timestamp shall be stored on the complaint.

FR-4.9.6: Clients shall supply device tilt. The mobile app samples it during in-app capture; the web client and gallery uploads do not, so the 35° default applies. *[Partial]*

FR-4.9.7: `POST /api/image-analysis/estimate-geometry` shall re-run estimation for a complaint and skip it with an explanatory message if the geometry columns are not migrated.

4.10 Administrative Workflows

Tools for triage, progress tracking and citizen management. **Priority:** High. **Status:** Implemented; assignment partial.

FR-4.10.1: `GET /api/admin/dashboard` and `GET /api/admin-enhanced/dashboard/overview` shall return complaint and user statistics, counts by status, resolution rate and the five highest-priority complaints.

FR-4.10.2: The priority queue (`GET /api/admin-enhanced/complaints/priority-queue`) shall order by priority, then newest first, and support free-text search, location, category, status, `flagged=true` and limit filters, returning citizen contact, authenticity, geometry and language fields where available.

FR-4.10.3: `PUT /api/admin/complaints/:id/status` shall accept `pending`, `in_progress`, `resolved` or `rejected` (else 400) with optional notes and admin ID, set `resolved_at` on resolution and return 404 for unknown complaints.

FR-4.10.4: `PUT /api/admin-enhanced/complaints/:id/stage/:stageId` shall update stage 1 (Initial Review), 2 (Assessment, with estimated cost) or 3 (Resolution) to `pending`, `in_progress`, `completed` or `cancelled`, log the change, and set the complaint to `resolved` when all stages are completed, `in_progress` when any stage is in progress, or `cancelled` when the stage is cancelled.

FR-4.10.5: A workflow record shall be created when an administrator first opens a complaint's details.

FR-4.10.6: Officers and contractors shall be assignable to stages. The API returns two hard-coded officers and two contractors, and stored assignee IDs are always null. *[Partial]*

FR-4.10.7: Administrators shall list and search citizens with complaint counts, view details, and delete a citizen together with their complaints and related records.

FR-4.10.8: The web complaint detail shall show the Explain AI view (FR-4.3.7) and the geometry estimate with a retry action (FR-4.9.7).

4.11 Transparency, Maps and Statistics

Public accountability views built from complaint data. **Priority:** Medium. **Status:** Implemented; heat-map weighting defect.

FR-4.11.1: `GET /api/transparency/dashboard` shall return totals by status, resolution rate, average resolution time, category statistics, a nine-month trend, impact and voting statistics, without authentication.

FR-4.11.2: `GET /api/heat-map/data` (default last 30 days; status and type filters) shall weight points by priority and by age of unresolved complaints (cumulative $\times 1.5$, $\times 2$, $\times 3$ after 7, 14 and 30 days). The priority bands assume a 0–100 scale while stored scores are 0–1, so the priority weighting is ineffective. *[Partial]*

FR-4.11.3: The web map shall render a Leaflet heat layer and circle markers on OpenStreetMap tiles; the mobile map shall use `react-native-maps` markers and circles.

FR-4.11.4: `GET /api/statistics/summary` shall return aggregate complaint statistics.

4.12 Civic Chatbot

An in-app assistant that explains how to use CITYZEN. **Priority:** Medium. **Status:** Implemented.

FR-4.12.1: `POST /api/chatbot/message` shall require a string `message` (else 400) and retrieve the best-matching knowledge-base topic using the last five messages as context.

FR-4.12.2: The reply shall be generated by Groq (default model `openai/gpt-oss-120b`, 8 s timeout), grounded in a summary of the whole knowledge base and the matched topic, in the requested one of six languages; on any Groq failure the knowledge-base reply shall be returned.

FR-4.12.3: Conversation sessions shall be kept in server memory per user ID, at most 100 sessions (oldest evicted), with history read and delete endpoints and an analytics endpoint.

4.13 Feedback, Localisation and Auxiliary Services

Priority: Low. **Status:** Implemented, except notifications.

FR-4.13.1: `POST /api/feedback/submit` shall require `complaintId` and a rating of 1–5 (else 400) for an existing complaint (else 404), and store it with the user ID or null; `GET /api/feedback/stats` shall return aggregates.

FR-4.13.2: Web and mobile UIs shall be available in English, Hindi, Tamil, Telugu, Kannada and Malayalam through i18next locale files.

FR-4.13.3: `GET /api/weather` shall return Open-Meteo current conditions and a daily forecast for the given coordinates (default Chennai) in the Asia/Kolkata time zone.

FR-4.13.4: Citizens and authorities shall be notified of priority or status changes. The submission response promises notification, but no notification channel exists. *[Not implemented]*

5. Non-Functional Requirements

No measured latency, throughput or availability figures exist; the requirements below state only limits that are enforced in code.

5.1 Performance

NFR-5.1.1: Every external AI call shall be time-bounded: Roboflow 20 s per attempt (max 3 attempts), zero-shot authenticity 7 s per call and 8 s overall, DistilBERT helper 10 s (health check 3 s), Groq 8 s, Sarvam 45 s.

NFR-5.1.2: Pothole geometry estimation shall not delay the submission response.

NFR-5.1.3: Duplicate detection shall precede text analysis and priority scoring so that duplicates do not incur those calls.

NFR-5.1.4: Responses shall be compressed; JSON bodies shall be limited to 10 MB and audio uploads to 50 MB.

5.2 Security and Privacy

NFR-5.2.1: Passwords shall be stored only as bcrypt hashes and never returned by the API.

NFR-5.2.2: The server shall refuse to start in production without `JWT_SECRET`.

NFR-5.2.3: The server shall set Helmet security headers and enforce the CORS allow-list in production.

NFR-5.2.4: Administrator endpoints, including citizen deletion, shall require an admin token (see FR-4.1.8). *[Not implemented]*

NFR-5.2.5: Creation of administrator accounts shall require verification; the admin sign-up is currently self-service. *[Not implemented]*

NFR-5.2.6: Credentials shall be supplied only through server environment variables. The Cloudinary cloud name and unsigned preset are hard-coded in both clients, a Maps API key is in `frontend/app/app.json`, and `POST /cloudinary/delete-image` is unauthenticated. *[Partial]*

NFR-5.2.7: Location shall be coarsened to the chosen privacy level before leaving the device; met on mobile, not on web (FR-4.6.3). *[Partial]*

NFR-5.2.8: Image URLs sent to Roboflow shall be HTTPS, and audio uploads shall be deleted after transcription.

5.3 Reliability and Availability

NFR-5.3.1: Unavailability of an AI service shall degrade, not fail, the affected function: authenticity falls back to rules, emotion to keywords (or 0.5), chatbot to the knowledge base, location scoring to 0.1, and geometry to status `failed`.

NFR-5.3.2: Optional columns (language, geometry, device ID) shall be probed and cached for 60 s so that the system keeps working before and after migrations without a restart.

NFR-5.3.3: Vote counts shall be recomputed from vote rows, not incremented, and the migrations shall enforce one vote per user or device per complaint.

NFR-5.3.4: Chatbot sessions and heat-map status caches are held in process memory and are lost on restart. *[Partial]*

5.4 Usability

NFR-5.4.1: The submission flow shall be divided into labelled steps and shall warn about description mismatch before the citizen submits.

NFR-5.4.2: The system shall explain its decisions: validation messages name the detected issue, and priority results include reasoning and next steps.

NFR-5.4.3: UI text, voice input and chatbot replies shall be available in all six languages.

NFR-5.4.4: Conformance to an accessibility standard (e.g. WCAG) has not been assessed. *[Unverified]*

5.5 Maintainability and Testability

NFR-5.5.1: All priority weights, thresholds and multipliers shall be defined once, with rationale, in `priorityConfig.js`; voting logic shall be shared through `voteService.js`; STT providers shall sit behind `SttProviderService`.

NFR-5.5.2: Pure geometry functions shall be unit-tested; all 13 Jest tests in `backend/services/potholeGeometryMath.test.js` pass. The Roboflow smoke test is skipped without credentials.

NFR-5.5.3: The web build shall run in CI on every push and pull request to `main`.

NFR-5.5.4: Unused legacy code (`adminEnhanced.js`, `adminSimplified.js`, empty route files, `.bak`/`.fixed` copies) shall be removed. *[Partial]*

5.6 Portability

NFR-5.6.1: The backend shall run on any Node.js 18+ host configured by environment variables; the web app shall build to static files; the mobile app shall target Android and iOS from one Expo code base.

NFR-5.6.2: The backend shall bind to `SERVER_HOST`/`PORT` (defaults 0.0.0.0 and 3001); the web client shall read `VITE_API_BASE_URL`, falling back to the Render URL.

6. Other Requirements

6.1 Glossary

Table 6.1: Glossary of Terms

Term	Definition
AHP	Analytic Hierarchy Process: derives normalised weights from pairwise 1–9 judgments (Saaty).
CR	Consistency Ratio of an AHP matrix; values ≤ 0.10 are acceptable.
SAM3	Segment Anything Model 3, a promptable segmentation model used inside the Roboflow workflows.
Roboflow Workflow	A hosted pipeline of vision steps invoked by HTTPS; CITYZEN uses one for validation and one for geometry.
Zero-shot NLI	Classifying text against arbitrary labels with a natural-language-inference model, without task-specific training.
STT	Speech-to-text.
JWT	JSON Web Token used as the bearer credential.
Homography	Projection of image points onto the road plane used to measure pothole footprints.
Privacy level	Exact, street (≈25 m) or area (≈150 m) coarsening of a complaint location.
Demo user	Shared account `demo@civicrezo.org` that owns anonymous submissions.
RLS	Row-Level Security policies in PostgreSQL/Supabase.

6.2 Data Dictionary

Columns of the first five tables are reconstructed from code (their base definitions are not available); the last two are defined by migrations in `backend/database`.

Table 6.2: CITYZEN Data Dictionary

Table	Principal columns and constraints
`users`	`id` (UUID), `email`, `password` (bcrypt hash), `full_name`, `phone_number`, `user_type` (citizen | admin), `address`, `is_active`, `created_at`, `last_login`.
`complaints` (core)	`id`, `user_id`, `title`, `description`, `category`, `status` (pending | in_progress | resolved | rejected | cancelled), `location_latitude`, `location_longitude`, `location_address`, `language` (default en), `image_urls`, `audio_url`, `vote_count`, `assigned_admin_id`, `resolution_notes`, `resolved_at`, timestamps.
`complaints` (analysis)	`priority_score`, `location_sensitivity_score`, `emotion_score`, `ai_confidence_score`, `text_authenticity_score` (2 decimals), `content_flagged`, `content_flag_reason`, `verification_status` (verified | unverified | flagged_for_review).
`complaints` (geometry)	`estimated_width_cm`, `estimated_length_cm`, `estimated_area_cm2`, `estimated_depth_cm`, `geometry_confidence`, `geometry_method`, `geometry_assumptions` (JSONB), `geometry_status` (pending | completed | failed | not_applicable), `geometry_error`, `geometry_computed_at`.
`complaint_votes`	`id`, `complaint_id`, `user_id`, `device_id`, `vote_type` (upvote), `created_at`; unique (complaint, user) and (complaint, device).
`complaint_updates`	`complaint_id`, `updated_by_id`, `old_status`, `new_status`, `update_notes`, `created_at`.

Table	Principal columns and constraints
`complaint_workflow`	`complaint_id`; for stages 1–3: `step_n_status`, `step_n_notes`, `step_n_timestamp`, assignee ID; `step_2_estimated_cost`, `step_3_completion_date`.
`complaint_feedback`	`id`, `complaint_id` (FK, cascade), `user_id` (nullable), `rating` (1–5, CHECK), `feedback_text`, `improvement_suggestions`, timestamps; RLS policies for own/guest/admin access.
`complaint_social_signals`	Schema only (platform, post, match and engagement scores, Roboflow cross-check fields); no code reads or writes it.

6.3 Open Issues and Future Enhancements

Table 6.3: Implementation Status of Incomplete or Unverified Items

Item	Status	Evidence and recommended action
Admin authorisation	Not implemented	`authorizeUserType` is unused.
Admin sign-up checks	Not implemented	Any visitor can sign up with `userType = admin`.
Server photo check	Not implemented	`/submit` trusts `imageValidation` from the client.
Priority recomputation	Not implemented	Vote and status terms fixed at submission.
Notifications	Not implemented	Next-step text promises it; no channel exists.
Social-media signals	Planned	Migration and X/Twitter env names only; no code.
Qwen STT provider	Planned	Stub throws “not configured yet”.
Staff assignment	Partial	Static lists; assignee IDs stored as null.
Web location privacy	Partial	Web sends exact coordinates with a privacy label.
Guest vote limits	Partial	Requires the `device_id` migration.
Heat-map weighting	Partial	Thresholds use a 0–100 scale; scores are 0–1.
Duplicate heuristic	Limitation	Distinct same-category issues within 40 m merge.
Base database schema	Unverified	No core-table DDL; `/:id/progress` joins tables whose definitions are unavailable.
Performance targets	Unverified	No benchmarks or SLAs; to be defined.
Web unit tests	Unverified	Vitest suites present; not run (dependencies absent).

6.4 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications.*
2. T. L. Saaty, *The Analytic Hierarchy Process*, McGraw-Hill, 1980.
3. T. L. Saaty, *Fundamentals of Decision Making and Priority Theory*, RWS Publications, 2000.
4. W. Luo and F. Wang, “Measures of spatial accessibility to health care in a GIS environment”, *Environment and Planning B*, 2003 (two-step floating catchment area method).
5. NFPA 1710, *Standard for the Organization and Deployment of Fire Suppression Operations, Emergency Medical Operations, and Special Operations to the Public by Career Fire Departments.*